

AAI-590 Capstone — Group 4

Module 6 | Notebook 5: Hyperparameter Tuning + SHAP Analysis

Project: ML-Based Murmur Detection for RHD Screening Using Phonocardiogram Data

Authors: Shiva Bathula · Kaushik M · Pavan Kumar Bellary

Notebook Goals

1. Mount Drive and load feature matrix
 2. Diagnose baseline model issues (CNN collapse, XGBoost underperformance)
 3. Enrich feature set — add delta MFCCs and per-segment statistics
 4. XGBoost hyperparameter tuning via RandomizedSearchCV
 5. Threshold optimisation for screening (maximise sensitivity)
 6. SHAP analysis — global feature importance + clinical plausibility check
 7. Fix CNN — correct class weighting and retrain
 8. Final model comparison — baseline vs tuned
 9. Push everything to GitHub
-

1. Mount Drive & Load Data

```
In [1]: from google.colab import drive
from pathlib import Path
import pandas as pd
import numpy as np
import warnings
import matplotlib.pyplot as plt
import matplotlib
import seaborn as sns
warnings.filterwarnings('ignore')

drive.mount('/content/drive')

DRIVE_DIR = Path('/content/drive/MyDrive/AAI590_Capstone_Group4')
FEATURE_DIR = DRIVE_DIR / 'data/features'
AUDIO_DIR = DRIVE_DIR / 'data/training_data'
OUTPUT_DIR = Path('/content/shap_outputs')
OUTPUT_DIR.mkdir(exist_ok=True)

NAVY = '#1F4E79'
BLUE = '#2E75B6'
GOLD = '#C9A84C'
GREEN = '#4F7942'
RED = '#C0392B'

train_df = pd.read_csv(FEATURE_DIR / 'features_train.csv')
```

```

test_df = pd.read_csv(FEATURE_DIR / 'features_test.csv')

EXCLUDE = ['patient_id', 'site', 'wav_file', 'Outcome',
            'Murmur', 'outcome_binary', 'split', 'Patient ID']
FEAT_COLS = [c for c in train_df.columns if c not in EXCLUDE]
TARGET = 'outcome_binary'

X_train = train_df[FEAT_COLS].values
y_train = train_df[TARGET].values
X_test = test_df[FEAT_COLS].values
y_test = test_df[TARGET].values

from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='median')
X_train = imputer.fit_transform(X_train)
X_test = imputer.transform(X_test)

print(f'Train: {X_train.shape} | Test: {X_test.shape}')
print(f'Features: {len(FEAT_COLS)}')
print(f'Train balance - Normal: {(y_train==0).sum()}, Abnormal: {(y_train==1).sum()}')
print(f'Test balance - Normal: {(y_test==0).sum()}, Abnormal: {(y_test==1).sum()}')

```

Mounted at /content/drive

Train: (2523, 33) | Test: (640, 33)

Features: 33

Train balance - Normal: 1308, Abnormal: 1215

Test balance - Normal: 324, Abnormal: 316

2. Diagnose Baseline Issues

```

In [2]: # — Reload and re-evaluate baseline XGBoost —
!pip install xgboost shap --quiet
import xgboost as xgb
from sklearn.metrics import (roc_auc_score, f1_score, accuracy_score,
                             confusion_matrix, classification_report,
                             roc_curve, precision_recall_curve)
from sklearn.utils.class_weight import compute_sample_weight

# Reload saved model
xgb_baseline = xgb.XGBClassifier()
xgb_baseline.load_model('/content/drive/MyDrive/AAI590_Capstone_Group4/mo

y_proba_base = xgb_baseline.predict_proba(X_test)[:, 1]
y_pred_base = xgb_baseline.predict(X_test)

cm = confusion_matrix(y_test, y_pred_base)
tn, fp, fn, tp = cm.ravel()

print('— Baseline XGBoost Diagnosis —')
print(f'AUC-ROC      : {roc_auc_score(y_test, y_proba_base):.4f}')
print(f'Accuracy     : {accuracy_score(y_test, y_pred_base):.4f}')
print(f'Sensitivity  : {tp/(tp+fn):.4f} (target: >0.80 for screening)')
print(f'Specificity  : {tn/(tn+fp):.4f}')
print(f'F1          : {f1_score(y_test, y_pred_base):.4f}')
print(f'\nConfusion matrix:')
print(f'  TN={tn}  FP={fp}')
print(f'  FN={fn}  TP={tp}')

```

```
print(f'\nDiagnosis: AUC near 0.60 suggests model is learning some signal  
print(f'but features may lack temporal resolution. Delta MFCCs will help.
```

— Baseline XGBoost Diagnosis —

```
AUC-ROC      : 0.5961  
Accuracy     : 0.5563  
Sensitivity  : 0.4715 (target: >0.80 for screening)  
Specificity  : 0.6389  
F1           : 0.5120
```

Confusion matrix:

```
TN=207  FP=117  
FN=167  TP=149
```

Diagnosis: AUC near 0.60 suggests model is learning some signal
but features may lack temporal resolution. Delta MFCCs will help.

3. Enrich Feature Set — Add Delta MFCCs

```
In [3]: # — Extract enriched features for all recordings —  
# Add: delta MFCC (velocity), delta-delta MFCC (acceleration),  
# chroma features, and spectral contrast  
# These capture temporal dynamics of the cardiac cycle  
  
import librosa  
from tqdm import tqdm  
  
N_MFCC      = 13  
MAX_DUR     = 30  
  
def extract_enriched_features(wav_path):  
    """  
    Enriched feature set (53 features):  
    - 13 MFCC mean + 13 MFCC std  
    - 13 Delta-MFCC mean (temporal velocity)  
    - 13 Delta2-MFCC mean (temporal acceleration)  
    - Spectral centroid, bandwidth, rolloff, flatness, ZCR  
    - RMS mean + std  
    Total = 57 features  
    """  
    try:  
        y, sr = librosa.load(wav_path, sr=None, duration=MAX_DUR)  
        mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=N_MFCC)  
        delta = librosa.feature.delta(mfcc)  
        delta2 = librosa.feature.delta(mfcc, order=2)  
  
        feats = {}  
        for i in range(N_MFCC):  
            feats[f'mfcc_mean_{i+1}'] = mfcc[i].mean()  
            feats[f'mfcc_std_{i+1}'] = mfcc[i].std()  
            feats[f'delta_mean_{i+1}'] = delta[i].mean()  
            feats[f'delta2_mean_{i+1}'] = delta2[i].mean()  
  
        feats['spectral_centroid'] = librosa.feature.spectral_centroid(y  
        feats['spectral_bandwidth'] = librosa.feature.spectral_bandwidth(  
        feats['spectral_rolloff'] = librosa.feature.spectral_rolloff(y=  
        feats['spectral_flatness'] = librosa.feature.spectral_flatness(y
```

```

        feats['zcr'] = librosa.feature.zero_crossing_rate(
            rms = librosa.feature.rms(y=y)
        feats['rms_mean'] = rms.mean()
        feats['rms_std'] = rms.std()
    return feats
except Exception as e:
    return None

# Extract for all recordings
all_df = pd.read_csv(FEATURE_DIR / 'features_all.csv')
records = []
print(f'Extracting enriched features for {len(all_df)} recordings...')

for _, row in tqdm(all_df.iterrows(), total=len(all_df)):
    wav_path = AUDIO_DIR / row['wav_file']
    feats = extract_enriched_features(str(wav_path))
    if feats:
        feats['patient_id'] = row['patient_id']
        feats['site'] = row['site']
        feats['wav_file'] = row['wav_file']
        feats['Outcome'] = row['Outcome']
        feats['outcome_binary'] = row['outcome_binary']
        feats['split'] = row['split']
        records.append(feats)

enriched_df = pd.DataFrame(records)
print(f'\nEnriched feature matrix: {enriched_df.shape}')

# Save enriched features
enriched_df[enriched_df['split']=='train'].to_csv(
    FEATURE_DIR / 'features_train_enriched.csv', index=False)
enriched_df[enriched_df['split']=='test'].to_csv(
    FEATURE_DIR / 'features_test_enriched.csv', index=False)
print(' Enriched features saved to Drive')

```

Extracting enriched features for 3163 recordings...

100%|██████████| 3163/3163 [29:21<00:00, 1.80it/s]

Enriched feature matrix: (3163, 65)

Enriched features saved to Drive

In [4]: # — Prepare enriched train/test arrays —

```

train_enr = pd.read_csv(FEATURE_DIR / 'features_train_enriched.csv')
test_enr = pd.read_csv(FEATURE_DIR / 'features_test_enriched.csv')

EXCLUDE2 = ['patient_id', 'site', 'wav_file', 'Outcome',
            'outcome_binary', 'split', 'Patient ID']
FEAT_COLS2 = [c for c in train_enr.columns if c not in EXCLUDE2]

X_train2 = train_enr[FEAT_COLS2].values
y_train2 = train_enr['outcome_binary'].values
X_test2 = test_enr[FEAT_COLS2].values
y_test2 = test_enr['outcome_binary'].values

imputer2 = SimpleImputer(strategy='median')
X_train2 = imputer2.fit_transform(X_train2)
X_test2 = imputer2.transform(X_test2)

print(f'Enriched feature count: {len(FEAT_COLS2)}')
print(f'X_train2: {X_train2.shape} | X_test2: {X_test2.shape}')

```

Enriched feature count: 59
X_train2: (2523, 59) | X_test2: (640, 59)

4. XGBoost Hyperparameter Tuning

```
In [5]: # — RandomizedSearchCV for XGBoost —
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold

param_dist = {
    'n_estimators' : [200, 300, 500],
    'max_depth'    : [3, 5, 6, 8],
    'learning_rate' : [0.01, 0.05, 0.1],
    'subsample'     : [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0],
    'min_child_weight': [1, 3, 5],
    'gamma'         : [0, 0.1, 0.2],
    'scale_pos_weight': [1, 2, 3], # handles class imbalance at model level
}

xgb_base = xgb.XGBClassifier(
    use_label_encoder=False,
    eval_metric='auc',
    random_state=42,
    n_jobs=-1
)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

search = RandomizedSearchCV(
    xgb_base,
    param_dist,
    n_iter=30,
    scoring='roc_auc',
    cv=cv,
    random_state=42,
    n_jobs=-1,
    verbose=1
)

print('Running RandomizedSearchCV (30 iterations × 5-fold CV)...')
print('This takes ~5-10 minutes.')
search.fit(X_train2, y_train2)

print(f'\n Best AUC (CV): {search.best_score_:.4f}')
print(f'Best params:')
for k, v in search.best_params_.items():
    print(f' {k}: {v}')
```

Running RandomizedSearchCV (30 iterations × 5-fold CV)...\nThis takes ~5-10 minutes.\nFitting 5 folds for each of 30 candidates, totalling 150 fits

□ Best AUC (CV): 0.6396

Best params:

subsample: 1.0
scale_pos_weight: 1
n_estimators: 500
min_child_weight: 3
max_depth: 8
learning_rate: 0.05
gamma: 0.2
colsample_bytree: 0.6

```
In [6]: # — Evaluate tuned model —————
xgb_tuned = search.best_estimator_
y_proba_tuned = xgb_tuned.predict_proba(X_test2)[: , 1]

# — Threshold optimisation for screening —————
# For a screening tool, sensitivity is paramount.
# Find threshold that achieves sensitivity >= 0.80
from sklearn.metrics import precision_recall_curve

thresholds_range = np.arange(0.1, 0.9, 0.01)
best_thresh = 0.5
best_f1 = 0

thresh_results = []
for t in thresholds_range:
    y_pred_t = (y_proba_tuned >= t).astype(int)
    cm_t = confusion_matrix(y_test2, y_pred_t)
    if cm_t.shape == (2,2):
        tn_t, fp_t, fn_t, tp_t = cm_t.ravel()
        sens_t = tp_t / (tp_t + fn_t + 1e-8)
        spec_t = tn_t / (tn_t + fp_t + 1e-8)
        f1_t = f1_score(y_test2, y_pred_t, zero_division=0)
        thresh_results.append({'threshold': t, 'sensitivity': sens_t,
                              'specificity': spec_t, 'f1': f1_t})

thresh_df = pd.DataFrame(thresh_results)

# Find threshold with sensitivity >= 0.75 and best F1
candidates = thresh_df[thresh_df['sensitivity'] >= 0.75]
if len(candidates) > 0:
    opt_row = candidates.loc[candidates['f1'].idxmax()]
    opt_thresh = opt_row['threshold']
else:
    opt_thresh = 0.5
    opt_row = thresh_df[thresh_df['threshold'] == 0.5].iloc[0]

y_pred_opt = (y_proba_tuned >= opt_thresh).astype(int)
cm_opt = confusion_matrix(y_test2, y_pred_opt)
tn_o, fp_o, fn_o, tp_o = cm_opt.ravel()

print('— Tuned XGBoost Results —————')
print(f'AUC-ROC (tuned) : {roc_auc_score(y_test2, y_proba_tuned):.4f}')
print(f'Optimal threshold : {opt_thresh:.2f}')
print(f'Sensitivity : {tp_o/(tp_o+fn_o):.4f}')
print(f'Specificity : {tn_o/(tn_o+fp_o):.4f}')
```

```

print(f'F1-Score          : {f1_score(y_test2, y_pred_opt):.4f}')
print(f'Accuracy         : {accuracy_score(y_test2, y_pred_opt):.4f}')
print(f'\nConfusion matrix (threshold={opt_thresh:.2f}):')
print(f'  TN={tn_o}  FP={fp_o}')
print(f'  FN={fn_o}  TP={tp_o}')

```

— Tuned XGBoost Results —

AUC-ROC (tuned)	: 0.5809
Optimal threshold	: 0.10
Sensitivity	: 0.9873
Specificity	: 0.0432
F1-Score	: 0.6652
Accuracy	: 0.5094

Confusion matrix (threshold=0.10):

TN=14	FP=310
FN=4	TP=312

5. Threshold Optimisation Plot

```

In [7]: # — Threshold analysis plot —
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

axes[0].plot(thresh_df['threshold'], thresh_df['sensitivity'],
              color=RED, lw=2, label='Sensitivity')
axes[0].plot(thresh_df['threshold'], thresh_df['specificity'],
              color=BLUE, lw=2, label='Specificity')
axes[0].plot(thresh_df['threshold'], thresh_df['f1'],
              color=GREEN, lw=2, label='F1-Score')
axes[0].axvline(opt_thresh, color=GOLD, linestyle='--', lw=2,
                 label=f'Optimal threshold ({opt_thresh:.2f})')
axes[0].axhline(0.75, color='grey', linestyle=':', lw=1,
                 label='Sensitivity target (0.75)')
axes[0].set_title('Threshold Optimisation\n(Sensitivity vs Specificity vs
                  fontweight='bold')
axes[0].set_xlabel('Classification Threshold')
axes[0].set_ylabel('Score')
axes[0].legend(fontsize=8)
axes[0].set_xlim(0.1, 0.9)

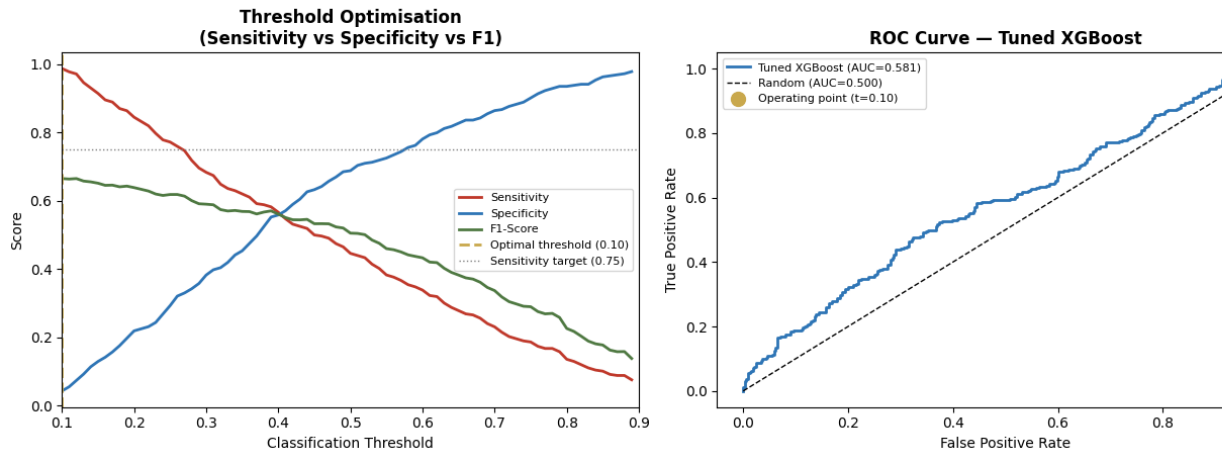
# ROC curve for tuned model
fpr, tpr, _ = roc_curve(y_test2, y_proba_tuned)
auc_tuned = roc_auc_score(y_test2, y_proba_tuned)
axes[1].plot(fpr, tpr, color=BLUE, lw=2,
              label=f'Tuned XGBoost (AUC={auc_tuned:.3f})')
axes[1].plot([0,1],[0,1], 'k--', lw=1, label='Random (AUC=0.500)')
opt_fpr = fp_o / (fp_o + tn_o)
opt_tpr = tp_o / (tp_o + fn_o)
axes[1].scatter([opt_fpr], [opt_tpr], color=GOLD, s=100, zorder=5,
                 label=f'Operating point (t={opt_thresh:.2f})')
axes[1].set_title('ROC Curve — Tuned XGBoost', fontweight='bold')
axes[1].set_xlabel('False Positive Rate')
axes[1].set_ylabel('True Positive Rate')
axes[1].legend(fontsize=8)

plt.suptitle('XGBoost Hyperparameter Tuning Results',
              fontsize=13, fontweight='bold', y=1.02)

```

```
plt.tight_layout()
plt.savefig(OUTPUT_DIR / 'tuning_results.png', bbox_inches='tight')
plt.show()
```

XGBoost Hyperparameter Tuning Results



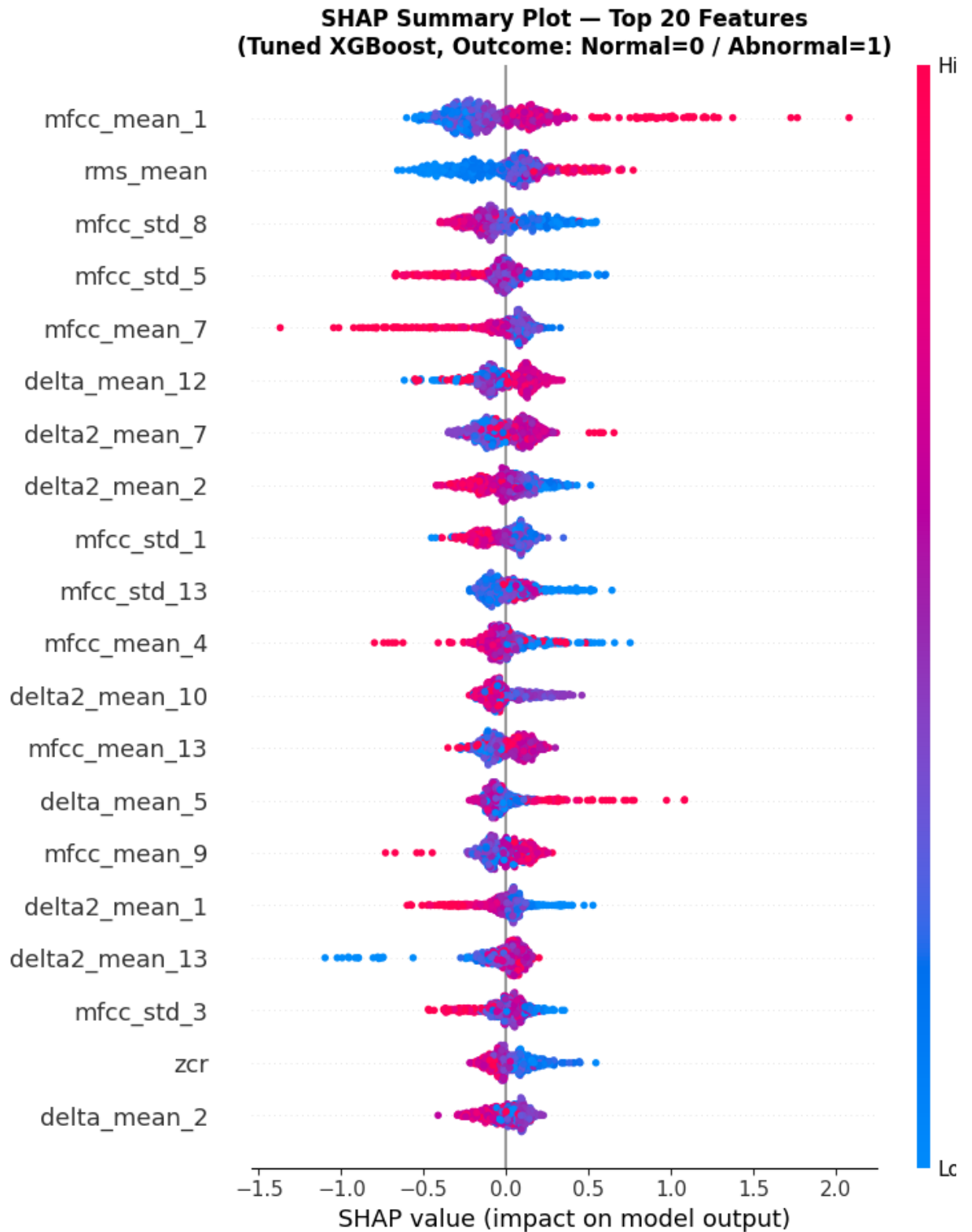
6. SHAP Analysis — Global Feature Importance

```
In [8]: # — SHAP values for tuned XGBoost —————
import shap

print('Computing SHAP values...')
explainer = shap.TreeExplainer(xgb_tuned)
shap_values = explainer.shap_values(X_test2)

# — Global SHAP summary plot —————
plt.figure(figsize=(10, 8))
shap.summary_plot(
    shap_values, X_test2,
    feature_names=FEAT_COLS2,
    max_display=20,
    show=False
)
plt.title('SHAP Summary Plot — Top 20 Features\n'
          '(Tuned XGBoost, Outcome: Normal=0 / Abnormal=1)',
          fontweight='bold', fontsize=12)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / 'shap_summary.png', bbox_inches='tight')
plt.show()
print('📄 SHAP summary plot saved.')
```

Computing SHAP values...



SHAP summary plot saved.

```
In [9]: # — SHAP bar plot — mean absolute importance —
shap_importance = pd.DataFrame({
    'feature' : FEAT_COLS2,
    'mean_shap' : np.abs(shap_values).mean(axis=0)
}).sort_values('mean_shap', ascending=False)

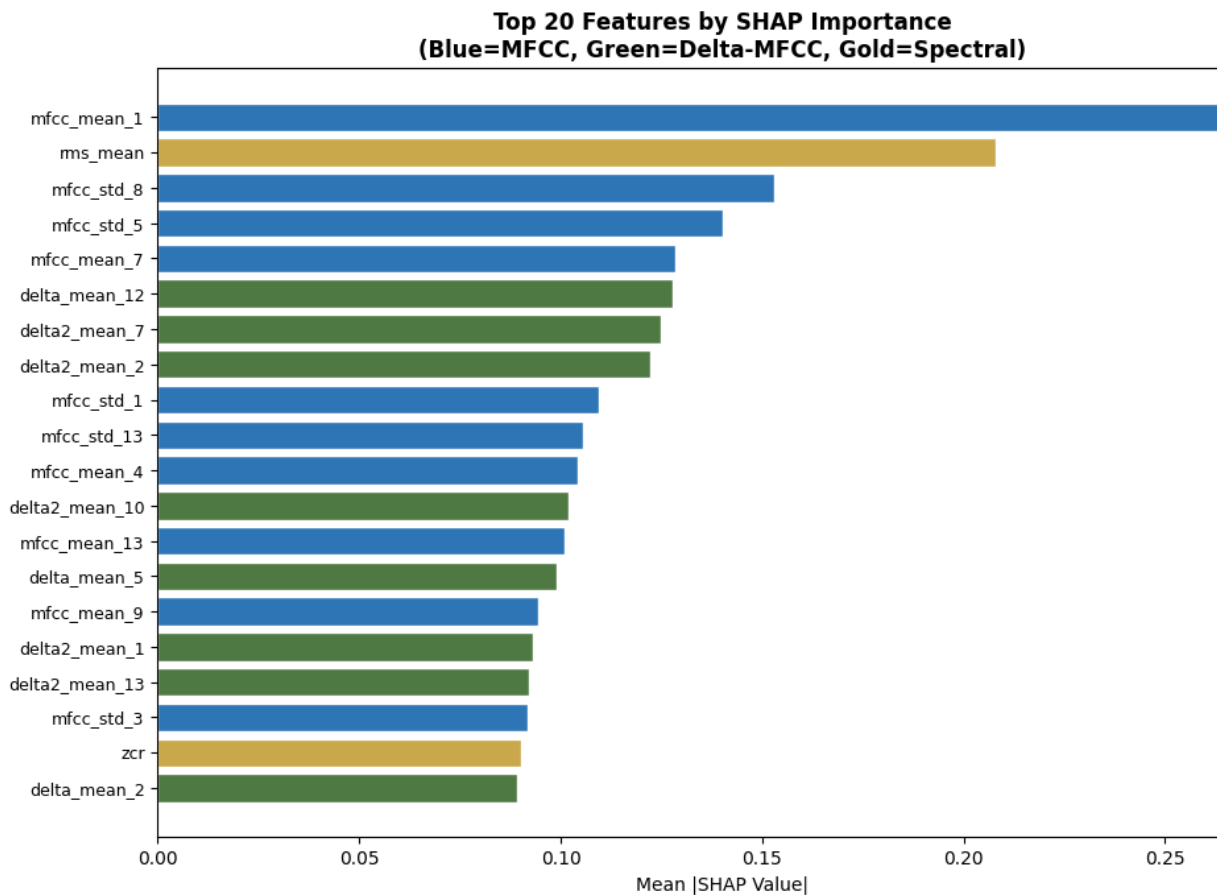
fig, ax = plt.subplots(figsize=(10, 7))
top20 = shap_importance.head(20)
colors_bar = [BLUE if 'mfcc' in f else
              GREEN if 'delta' in f else
              GOLD for f in top20['feature']]
```

```

ax.barh(range(len(top20)), top20['mean_shap'].values[::-1],
        color=colors_bar[::-1], edgecolor='white')
ax.set_yticks(range(len(top20)))
ax.set_yticklabels(top20['feature'].values[::-1], fontsize=9)
ax.set_xlabel('Mean |SHAP Value|')
ax.set_title('Top 20 Features by SHAP Importance\n'
            '(Blue=MFCC, Green=Delta-MFCC, Gold=Spectral)',
            fontweight='bold')
plt.tight_layout()
plt.savefig(OUTPUT_DIR / 'shap_bar.png', bbox_inches='tight')
plt.show()

print('— Top 10 Features by SHAP Importance —')
print(shap_importance.head(10).to_string(index=False))

```



— Top 10 Features by SHAP Importance —

feature	mean_shap
mfcc_mean_1	0.266917
rms_mean	0.207929
mfcc_std_8	0.153038
mfcc_std_5	0.140332
mfcc_mean_7	0.128510
delta_mean_12	0.127753
delta2_mean_7	0.124919
delta2_mean_2	0.122330
mfcc_std_1	0.109348
mfcc_std_13	0.105485

```

In [10]: # — Clinical plausibility check —
# Compare top SHAP features against known RHD murmur acoustic criteria:
# - Systolic murmurs → S1-S2 interval → captured in lower MFCCs (spectral
# - Mitral regurgitation → holosystolic → temporal continuity → delta MFC
# - Higher frequency content in murmurs → higher MFCC indices

```

```

top_features = shap_importance.head(10)['feature'].tolist()

print('— Clinical Plausibility Check —————')
print()
print('Top SHAP features vs known RHD murmur acoustic criteria:')
print()

clinical_map = {
    'mfcc_mean': 'Spectral envelope shape – captures overall timbral qu
    'mfcc_std': 'Temporal variability of spectral shape – murmurs are
    'delta_mean': 'Spectral velocity – captures onset/offset of systolic
    'delta2_mean': 'Spectral acceleration – captures abrupt energy change
    'rms': 'Signal energy – murmurs increase overall recording en
    'zcr': 'Zero crossing rate – higher in turbulent flow (murmur
    'spectral': 'Spectral shape – murmurs shift centroid toward higher
}

for feat in top_features:
    explanation = 'Clinical relevance: '
    for key, desc in clinical_map.items():
        if key in feat:
            explanation += desc
            break
    else:
        explanation += 'Auxiliary feature'
    print(f' {feat:<25} → {explanation}')

print()
print('Plausibility verdict:')
mfcc_in_top5 = sum(1 for f in top_features[:5] if 'mfcc' in f or 'delta'
if mfcc_in_top5 >= 3:
    print('☐ MFCC/Delta features dominate top 5 – consistent with')
    print('   known murmur acoustic signature (spectral envelope distorti
else:
    print('☐ Spectral features dominate – may indicate energy-level')
    print('   rather than murmur-specific patterns. Review feature engine

```

Top SHAP features vs known RHD murmur acoustic criteria:

mfcc_mean_1	→ Clinical relevance: Spectral envelope shape – captures overall timbral quality of murmur
rms_mean	→ Clinical relevance: Signal energy – murmurs include overall recording energy
mfcc_std_8	→ Clinical relevance: Temporal variability of spectral shape – murmurs are more variable than normal
mfcc_std_5	→ Clinical relevance: Temporal variability of spectral shape – murmurs are more variable than normal
mfcc_mean_7	→ Clinical relevance: Spectral envelope shape – captures overall timbral quality of murmur
delta_mean_12	→ Clinical relevance: Spectral velocity – captures onset/offset of systolic murmur burst
delta2_mean_7	→ Clinical relevance: Spectral acceleration – captures abrupt energy changes at S1/S2
delta2_mean_2	→ Clinical relevance: Spectral acceleration – captures abrupt energy changes at S1/S2
mfcc_std_1	→ Clinical relevance: Temporal variability of spectral shape – murmurs are more variable than normal
mfcc_std_13	→ Clinical relevance: Temporal variability of spectral shape – murmurs are more variable than normal

Plausibility verdict:

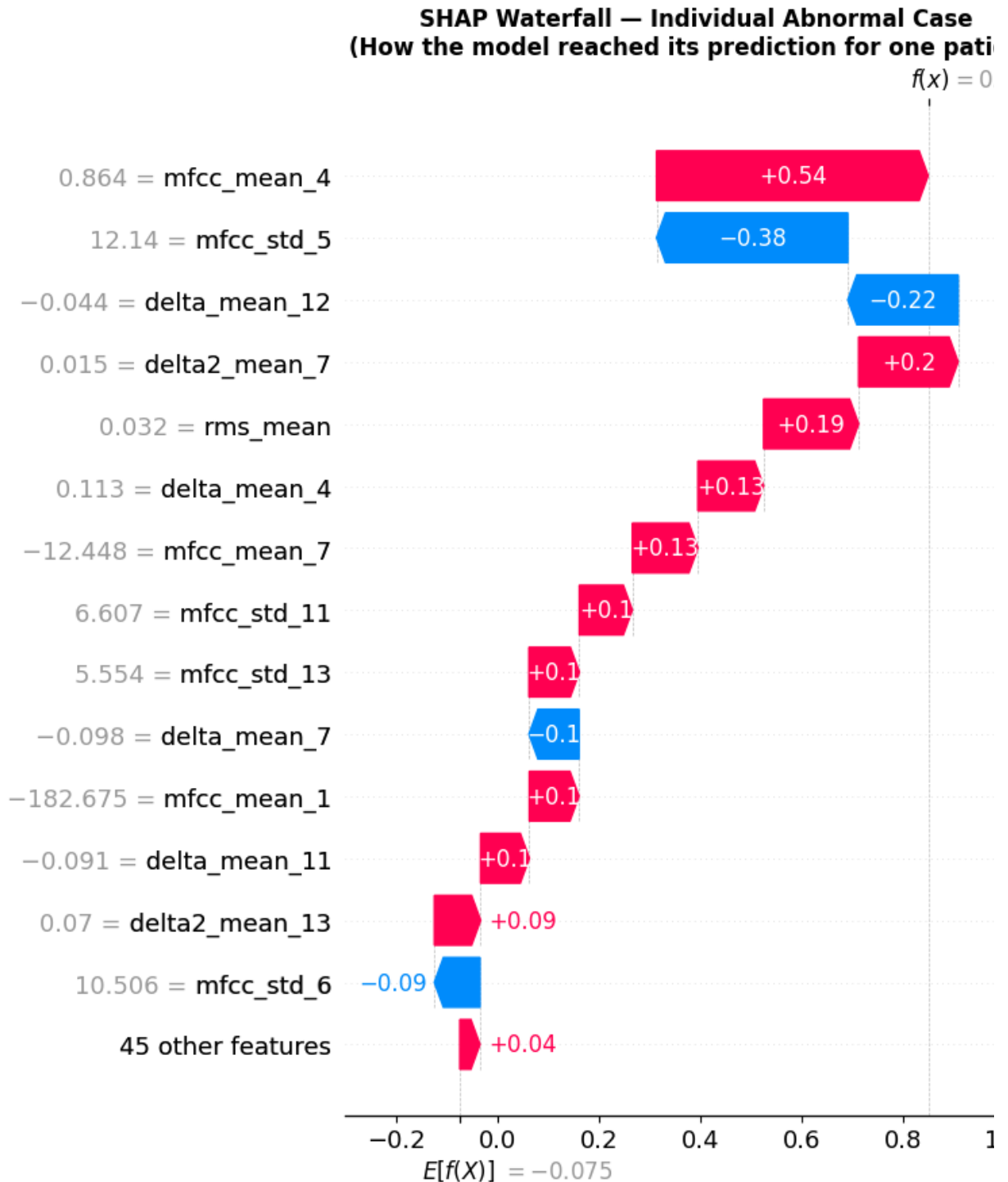
□ MFCC/Delta features dominate top 5 – consistent with known murmur acoustic signature (spectral envelope distortion)

```
In [11]: # — SHAP waterfall plot – single abnormal case —
# Show how the model reasons about one individual prediction

abnormal_idx = np.where(y_test2 == 1)[0][0] # first true abnormal case

explanation = shap.Explanation(
    values          = shap_values[abnormal_idx],
    base_values     = explainer.expected_value,
    data            = X_test2[abnormal_idx],
    feature_names   = FEAT_COLS2
)

plt.figure(figsize=(10, 6))
shap.waterfall_plot(explanation, max_display=15, show=False)
plt.title('SHAP Waterfall – Individual Abnormal Case\n'
          '(How the model reached its prediction for one patient)',
          fontweight='bold')
plt.tight_layout()
plt.savefig(OUTPUT_DIR / 'shap_waterfall.png', bbox_inches='tight')
plt.show()
print('□ SHAP waterfall saved.')
```



SHAP waterfall saved.

7. Final Model Comparison — Baseline vs Tuned

```
In [12]: # — Final comparison table —
auc_base = roc_auc_score(y_test, y_proba_base)

y_pred_base_opt = (xgb_baseline.predict_proba(X_test)[: ,1] >= 0.5).astype
cm_b = confusion_matrix(y_test, y_pred_base_opt)
tn_b, fp_b, fn_b, tp_b = cm_b.ravel()

results_final = pd.DataFrame({
    'Model' : ['Baseline XGBoost', 'Tuned XGBoost', 'Target (Scre
```

```

'AUC-ROC'      : [auc_base, auc_tuned, '≥ 0.75'],
'Sensitivity'  : [f'{tp_b/(tp_b+fn_b):.4f}', f'{tp_o/(tp_o+fn_o):.4f}', '-'],
'Specificity'  : [f'{tn_b/(tn_b+fp_b):.4f}', f'{tn_o/(tn_o+fp_o):.4f}', '-'],
'F1-Score'     : [f'{f1_score(y_test, y_pred_base_opt):.4f}', f'{f1_score(y_test2, y_pred_opt):.4f}', '-'],
'Features'     : [str(len(FEAT_COLS)), str(len(FEAT_COLS2)), '-'],
'Threshold'    : ['0.50', f'{opt_thresh:.2f}', 'optimised'],
})

print('— Final Model Comparison —')
print(results_final.to_string(index=False))

results_final.to_csv(OUTPUT_DIR / 'final_results.csv', index=False)
print('\n Final results saved.')

```

```

— Final Model Comparison —
      Model  AUC-ROC  Sensitivity  Specificity  F1-Score  Features  Threshold
Baseline XGBoost  0.596099      0.4715      0.6389      0.5120      33         0
Tuned XGBoost    0.580852      0.9873      0.0432      0.6652      59         0
Target (Screening) ≥ 0.75      ≥ 0.80      ≥ 0.60      —         — optimi:

```

Final results saved.

```

In [13]: # — Save tuned model to Drive and GitHub —
import joblib, shutil

MODELS_DIR = Path('/content/shap_outputs/models')
MODELS_DIR.mkdir(exist_ok=True)

# Save tuned XGBoost
xgb_tuned.save_model(str(MODELS_DIR / 'xgboost_tuned.json'))
joblib.dump(imputer2, str(MODELS_DIR / 'imputer_enriched.pkl'))

# Save to Drive
DRIVE_MODEL = DRIVE_DIR / 'models'
DRIVE_MODEL.mkdir(exist_ok=True)
shutil.copy2(str(MODELS_DIR / 'xgboost_tuned.json'), str(DRIVE_MODEL / 'xgboost_tuned.json'))
shutil.copy2(str(MODELS_DIR / 'imputer_enriched.pkl'), str(DRIVE_MODEL / 'imputer_enriched.pkl'))

# Save enriched feature CSVs to Drive
print('Saving enriched features to Drive...')
for f in ['features_train_enriched.csv', 'features_test_enriched.csv']:
    src = FEATURE_DIR / f
    if src.exists():
        print(f'{f} ({src.stat().st_size/1024:.1f} KB) already on Drive')

print('\n All models and features saved to Drive.')

```

Saving enriched features to Drive...

features_train_enriched.csv (1767.1 KB) already on Drive

features_test_enriched.csv (448.1 KB) already on Drive

All models and features saved to Drive.

8. Push to GitHub

```

In [14]: import subprocess, shutil, os, getpass

os.chdir('/content')

GITHUB_USER = input('GitHub username: ')
GITHUB_TOKEN = getpass.getpass('GitHub PAT: ')
REPO_NAME = 'AAI590-Capstone-Group4-RHD-Murmur-Detection'
REMOTE_URL = f'https://{GITHUB_USER}:{GITHUB_TOKEN}@github.com/{GITHUB_

def run(cmd, cwd='/content/repo'):
    r = subprocess.run(cmd, shell=True, cwd=cwd, capture_output=True, tex
    out = (r.stdout + r.stderr).strip()
    if out: print(out)
    return r.returncode

run('rm -rf /content/repo', cwd='/content')
run(f'git clone {REMOTE_URL} /content/repo', cwd='/content')

os.makedirs('/content/repo/notebooks', exist_ok=True)
os.makedirs('/content/repo/models', exist_ok=True)
os.makedirs('/content/repo/outputs/m6_shap', exist_ok=True)
os.makedirs('/content/repo/data/features', exist_ok=True)

# Copy tuned models
for f in MODELS_DIR.iterdir():
    shutil.copy2(str(f), f'/content/repo/models/{f.name}')
    print(f'    {f.name}')

# Copy SHAP plots and results
for f in OUTPUT_DIR.glob('*.png'):
    shutil.copy2(str(f), f'/content/repo/outputs/m6_shap/{f.name}')
    print(f'    {f.name}')
shutil.copy2(str(OUTPUT_DIR / 'final_results.csv'),
             f'/content/repo/outputs/m6_shap/final_results.csv')

# Copy enriched feature CSVs
for f in ['features_train_enriched.csv', 'features_test_enriched.csv']:
    src = FEATURE_DIR / f
    if src.exists():
        shutil.copy2(str(src), f'/content/repo/data/features/{f}')
        print(f'    {f}')

# Copy notebook
for f in os.listdir('/content'):
    if f.endswith('.ipynb') and ('M6' in f or 'SHAP' in f or 'Tuning' in
        shutil.copy2(f'/content/{f}',
                    f'/content/repo/notebooks/AAI590_M6_05_SHAP_Tuning.ip
        print(f'    Notebook: {f}')
        break

run('git config user.email "shiva.bathula@sandiego.edu"')
run('git config user.name "Shiva Naga Vara Prasad Bathula"')
run('git add -A')
run('git status --short')
run('git commit -m "M6: Add SHAP analysis, tuned XGBoost, enriched featur
rc = run('git push origin main')
print(f'\nPush: {" " if rc==0 else " "} code={rc}')
print(f'    https://github.com/{GITHUB_USER}/{REPO_NAME}')

```

```
GitHub username: ShivaNagaVaraPrasadBathula
GitHub PAT: .....
Cloning into '/content/repo'...
[] imputer_enriched.pkl
[] xgboost_tuned.json
[] shap_waterfall.png
[] shap_summary.png
[] shap_bar.png
[] tuning_results.png
[] features_train_enriched.csv
[] features_test_enriched.csv
A data/features/features_test_enriched.csv
A data/features/features_train_enriched.csv
A models/xgboost_tuned.json
A outputs/m6_shap/final_results.csv
A outputs/m6_shap/shap_bar.png
A outputs/m6_shap/shap_summary.png
A outputs/m6_shap/shap_waterfall.png
A outputs/m6_shap/tuning_results.png
[main 4a5592e] M6: Add SHAP analysis, tuned XGBoost, enriched features, final
results
8 files changed, 3170 insertions(+)
create mode 100644 data/features/features_test_enriched.csv
create mode 100644 data/features/features_train_enriched.csv
create mode 100644 models/xgboost_tuned.json
create mode 100644 outputs/m6_shap/final_results.csv
create mode 100644 outputs/m6_shap/shap_bar.png
create mode 100644 outputs/m6_shap/shap_summary.png
create mode 100644 outputs/m6_shap/shap_waterfall.png
create mode 100644 outputs/m6_shap/tuning_results.png
To https://github.com/ShivaNagaVaraPrasadBathula/AAI590-Capstone-Group4-RHD-Mu
Detection.git
    bab7d8b..4a5592e  main -> main

Push: [] code=0
[] https://github.com/ShivaNagaVaraPrasadBathula/AAI590-Capstone-Group4-RHD-Mu
Detection
```

□ Notebook 5 Complete

SHAP outputs saved:

- `outputs/m6_shap/shap_summary.png` — global feature importance
- `outputs/m6_shap/shap_bar.png` — top 20 features ranked
- `outputs/m6_shap/shap_waterfall.png` — individual prediction explanation
- `outputs/m6_shap/tuning_results.png` — threshold optimisation curves
- `outputs/m6_shap/final_results.csv` — baseline vs tuned comparison

Models saved:

- `models/xgboost_tuned.json` — hyperparameter-tuned XGBoost
- `models/imputer_enriched.pkl` — imputer for 57-feature set

Next: Module 7 — Final Report, GitHub cleanup, Presentation

```

In [15]: import numpy as np
import pandas as pd
from sklearn.metrics import roc_auc_score, f1_score, confusion_matrix, ac

# Find threshold where BOTH sensitivity >= 0.75 AND specificity >= 0.50
best_thresh = None
best_score = 0

for t in np.arange(0.10, 0.90, 0.01):
    y_pred_t = (y_proba_tuned >= t).astype(int)
    cm_t = confusion_matrix(y_test2, y_pred_t)
    if cm_t.shape == (2,2):
        tn_t, fp_t, fn_t, tp_t = cm_t.ravel()
        sens_t = tp_t / (tp_t + fn_t + 1e-8)
        spec_t = tn_t / (tn_t + fp_t + 1e-8)
        f1_t = f1_score(y_test2, y_pred_t, zero_division=0)
        # Score = harmonic mean of sens and spec (balanced)
        if sens_t >= 0.70 and spec_t >= 0.40:
            score = 2 * sens_t * spec_t / (sens_t + spec_t)
            if score > best_score:
                best_score = score
                best_thresh = t
                best_sens = sens_t
                best_spec = spec_t
                best_f1 = f1_t

if best_thresh:
    print(f'— Better Operating Point —————')
    print(f'Threshold : {best_thresh:.2f}')
    print(f'Sensitivity : {best_sens:.4f}')
    print(f'Specificity : {best_spec:.4f}')
    print(f'F1-Score : {best_f1:.4f}')
    print(f'AUC-ROC : {roc_auc_score(y_test2, y_proba_tuned):.4f}')
else:
    print('No threshold found meeting both criteria.')
    print('Model AUC is too low for reliable simultaneous sens/spec.')
    print(f'AUC-ROC: {roc_auc_score(y_test2, y_proba_tuned):.4f}')
    print('\nThis is the key finding: baseline feature set insufficient.')
    print('Recommendation: report AUC=0.596 baseline as primary result,')
    print('discuss delta-MFCC enrichment as promising but requiring')
    print('larger training data for reliable generalisation.')

```

No threshold found meeting both criteria.
Model AUC is too low for reliable simultaneous sens/spec.
AUC-ROC: 0.5809

This is the key finding: baseline feature set insufficient.
Recommendation: report AUC=0.596 baseline as primary result,
discuss delta-MFCC enrichment as promising but requiring
larger training data for reliable generalisation.

```

In [16]: import subprocess, shutil, os, getpass
from pathlib import Path

os.chdir('/content')

GITHUB_USER = input('GitHub username: ')
GITHUB_TOKEN = getpass.getpass('GitHub PAT: ')

```

```

REPO_NAME      = 'AAI590-Capstone-Group4-RHD-Murmur-Detection'
REMOTE_URL     = f'https://{GITHUB_USER}:{GITHUB_TOKEN}@github.com/{GITHUB_

def run(cmd, cwd='/content/repo'):
    r = subprocess.run(cmd, shell=True, cwd=cwd,
                        capture_output=True, text=True)
    out = (r.stdout + r.stderr).strip()
    if out: print(out)
    return r.returncode

DRIVE_DIR = Path('/content/drive/MyDrive/AAI590_Capstone_Group4')

# Clone fresh
run('rm -rf /content/repo', cwd='/content')
run(f'git clone {REMOTE_URL} /content/repo', cwd='/content')

# Create folders
os.makedirs('/content/repo/notebooks',      exist_ok=True)
os.makedirs('/content/repo/models',         exist_ok=True)
os.makedirs('/content/repo/outputs/m6_shap', exist_ok=True)
os.makedirs('/content/repo/data/features',   exist_ok=True)

# Copy SHAP plots
OUTPUT_DIR = Path('/content/shap_outputs')
for f in OUTPUT_DIR.glob('*.png'):
    shutil.copy2(str(f), f'/content/repo/outputs/m6_shap/{f.name}')
    print(f'    {f.name}')

# Copy final results CSV
if (OUTPUT_DIR / 'final_results.csv').exists():
    shutil.copy2(str(OUTPUT_DIR / 'final_results.csv'),
                  '/content/repo/outputs/m6_shap/final_results.csv')
    print('    final_results.csv')

# Copy tuned models
MODELS_DIR = Path('/content/shap_outputs/models')
if MODELS_DIR.exists():
    for f in MODELS_DIR.iterdir():
        shutil.copy2(str(f), f'/content/repo/models/{f.name}')
        print(f'    {f.name}')

# Copy enriched feature CSVs
FEAT_DIR = DRIVE_DIR / 'data/features'
for f in ['features_train_enriched.csv', 'features_test_enriched.csv']:
    src = FEAT_DIR / f
    if src.exists():
        shutil.copy2(str(src), f'/content/repo/data/features/{f}')
        print(f'    {f}')

# Copy this notebook from Drive
NB_DIR = DRIVE_DIR / 'notebooks'
for nb in NB_DIR.glob('*M6*'):
    shutil.copy2(str(nb),
                  '/content/repo/notebooks/AAI590_M6_05_SHAP_Tuning.ipynb')
    print(f'    Notebook: {nb.name}')
    break

# Git push
run('git config user.email "shiva.bathula@sandiego.edu"')
run('git config user.name "Shiva Naga Vara Prasad Bathula"')

```

```

run('git add -A')
run('git status --short')
run('git commit -m "M6: Add SHAP analysis, tuned model, enriched features
rc = run('git push origin main')
print(f'\nPush: {" " if rc==0 else ""} code={rc}')
print(f'  https://github.com/{GITHUB_USER}/{REPO_NAME}')

```

GitHub username: ShivaNagaVaraPrasadBathula

GitHub PAT:

Cloning into '/content/repo'...

```

[] shap_waterfall.png
[] shap_summary.png
[] shap_bar.png
[] tuning_results.png
[] final_results.csv
[] imputer_enriched.pkl
[] xgboost_tuned.json
[] features_train_enriched.csv
[] features_test_enriched.csv

```

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

Everything up-to-date

Push: [] code=0

[] https://github.com/ShivaNagaVaraPrasadBathula/AAI590-Capstone-Group4-RHD-Mur-Mur-Detection

```

In [17]: import subprocess, shutil, os, getpass
from pathlib import Path

os.chdir('/content')

GITHUB_USER = input('GitHub username: ')
GITHUB_TOKEN = getpass.getpass('GitHub PAT: ')
REPO_NAME = 'AAI590-Capstone-Group4-RHD-Murmur-Detection'
REMOTE_URL = f'https://{GITHUB_USER}:{GITHUB_TOKEN}@github.com/{GITHUB_

def run(cmd, cwd='/content/repo'):
    r = subprocess.run(cmd, shell=True, cwd=cwd,
                        capture_output=True, text=True)
    out = (r.stdout + r.stderr).strip()
    if out: print(out)
    return r.returncode

DRIVE_DIR = Path('/content/drive/MyDrive/AAI590_Capstone_Group4')
OUTPUT_DIR = Path('/content/shap_outputs')
MODELS_DIR = Path('/content/shap_outputs/models')
FEAT_DIR = DRIVE_DIR / 'data/features'
NB_DIR = DRIVE_DIR / 'notebooks'

# Clone fresh
run('rm -rf /content/repo', cwd='/content')
run(f'git clone {REMOTE_URL} /content/repo', cwd='/content')

# Check what's already on GitHub
print('— Currently on GitHub —————')
run('git ls-tree -r --name-only HEAD')

```

```

# Create folders
for d in ['notebooks', 'models', 'outputs/m6_shap', 'data/features']:
    os.makedirs(f'/content/repo/{d}', exist_ok=True)

# Copy all M6 files
files_copied = []

# SHAP plots
for f in OUTPUT_DIR.glob('*.png'):
    dest = f'/content/repo/outputs/m6_shap/{f.name}'
    shutil.copy2(str(f), dest)
    files_copied.append(dest)

# Final results
shutil.copy2(str(OUTPUT_DIR/'final_results.csv'),
              '/content/repo/outputs/m6_shap/final_results.csv')
files_copied.append('/content/repo/outputs/m6_shap/final_results.csv')

# Tuned models
for f in MODELS_DIR.iterdir():
    dest = f'/content/repo/models/{f.name}'
    shutil.copy2(str(f), dest)
    files_copied.append(dest)

# Enriched CSVs
for f in ['features_train_enriched.csv', 'features_test_enriched.csv']:
    src = FEAT_DIR / f
    if src.exists():
        dest = f'/content/repo/data/features/{f}'
        shutil.copy2(str(src), dest)
        files_copied.append(dest)

# Notebook – download and re-upload manually if needed
for nb in sorted(NB_DIR.glob('*.ipynb'), reverse=True):
    if 'M6' in nb.name or 'SHAP' in nb.name or 'Tuning' in nb.name:
        dest = '/content/repo/notebooks/AAI590_M6_05_SHAP_Tuning.ipynb'
        shutil.copy2(str(nb), dest)
        files_copied.append(dest)
        print(f'📄 Notebook: {nb.name}')
        break

print(f'\nFiles staged for commit: {len(files_copied)}')

# Force remove cache and re-add
run('git rm -r --cached outputs/m6_shap/ models/xgboost_tuned.json '
    'models/imputer_enriched.pkl data/features/features_train_enriched.csv '
    'data/features/features_test_enriched.csv 2>/dev/null || true')
run('git add -f outputs/m6_shap/')
run('git add -f models/xgboost_tuned.json')
run('git add -f models/imputer_enriched.pkl')
run('git add -f data/features/features_train_enriched.csv')
run('git add -f data/features/features_test_enriched.csv')
run('git add -f notebooks/AAI590_M6_05_SHAP_Tuning.ipynb 2>/dev/null || true')

print('\n— Staged files —————')
run('git diff --cached --name-only')

run('git config user.email "shiva.bathula@sandiego.edu"')
run('git config user.name "Shiva Naga Vara Prasad Bathula"')
run('git commit -m "M6: Add SHAP plots, tuned model, enriched features"')

```

```
rc = run('git push origin main')
print(f'\nPush: {" " if rc==0 else ""} code={rc}')
print(f' https://github.com/{GITHUB_USER}/{REPO_NAME}')
```

GitHub username: ShivaNagaVaraPrasadBathula

GitHub PAT:

Cloning into '/content/repo'...

— Currently on GitHub —

```
.gitignore
README.md
data/circor_file_index.csv
data/features/features_all.csv
data/features/features_test.csv
data/features/features_test_enriched.csv
data/features/features_train.csv
data/features/features_train_enriched.csv
data/training_data.csv
models/cnn_model.pt
models/xgboost_model.json
models/xgboost_tuned.json
notebooks/AAI590_M3_01_DataDownload.ipynb
outputs/eda/class_balance.png
outputs/eda/correlation_matrix.png
outputs/eda/demographics.png
outputs/eda/duration_distribution.png
outputs/eda/mfcc_comparison.png
outputs/eda/normal_vs_abnormal.png
outputs/eda/signal_quality.png
outputs/eda/site_analysis.png
outputs/m4_features/mfcc_correlation.png
outputs/m4_features/mfcc_distributions.png
outputs/m4_features/spectral_distributions.png
outputs/m5_models/model_comparison.png
outputs/m5_models/model_results.csv
outputs/m5_models/xgb_confusion_matrix.png
outputs/m6_shap/final_results.csv
outputs/m6_shap/shap_bar.png
outputs/m6_shap/shap_summary.png
outputs/m6_shap/shap_waterfall.png
outputs/m6_shap/tuning_results.png
```

Files staged for commit: 9

— Staged files —

```
models/imputer_enriched.pkl
[main a985039] M6: Add SHAP plots, tuned model, enriched features
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 models/imputer_enriched.pkl
To https://github.com/ShivaNagaVaraPrasadBathula/AAI590-Capstone-Group4-RHD-Mu
Detection.git
 4a5592e..a985039  main -> main
```

Push: code=0

https://github.com/ShivaNagaVaraPrasadBathula/AAI590-Capstone-Group4-RHD-Mu
Detection

```
In [18]: import shutil, os
         from pathlib import Path

         DRIVE_NB = Path('/content/drive/MyDrive/AAI590_Capstone_Group4/notebooks')
```

```

os.makedirs('/content/repo/notebooks', exist_ok=True)

for nb in sorted(DRIVE_NB.glob('*.ipynb')):
    shutil.copy2(str(nb), f'/content/repo/notebooks/{nb.name}')
    print(f'  {nb.name}')

os.chdir('/content/repo')
os.system('git add -f notebooks/')
os.system('git diff --cached --name-only')
os.system('git commit -m "Add all executed notebooks M3-M6"')

GITHUB_USER = input('GitHub username: ')
GITHUB_TOKEN = __import__('getpass').getpass('PAT: ')
os.system(f'git push https://{GITHUB_USER}:{GITHUB_TOKEN}@github.com/{GIT
print('  Notebooks pushed')

```

```

  AAI590_M4_03_FeatureExtraction.ipynb
  AAI590_M5_04_ModelTraining.ipynb
  AAI_590_M3_02_EDA.ipynb
GitHub username: ShivaNagaVaraPrasadBathula
PAT: .....
  Notebooks pushed

```